P, NP, CoNP, NP hard and NP complete | Complexity Classes

Last Updated : 22 Feb, 2025

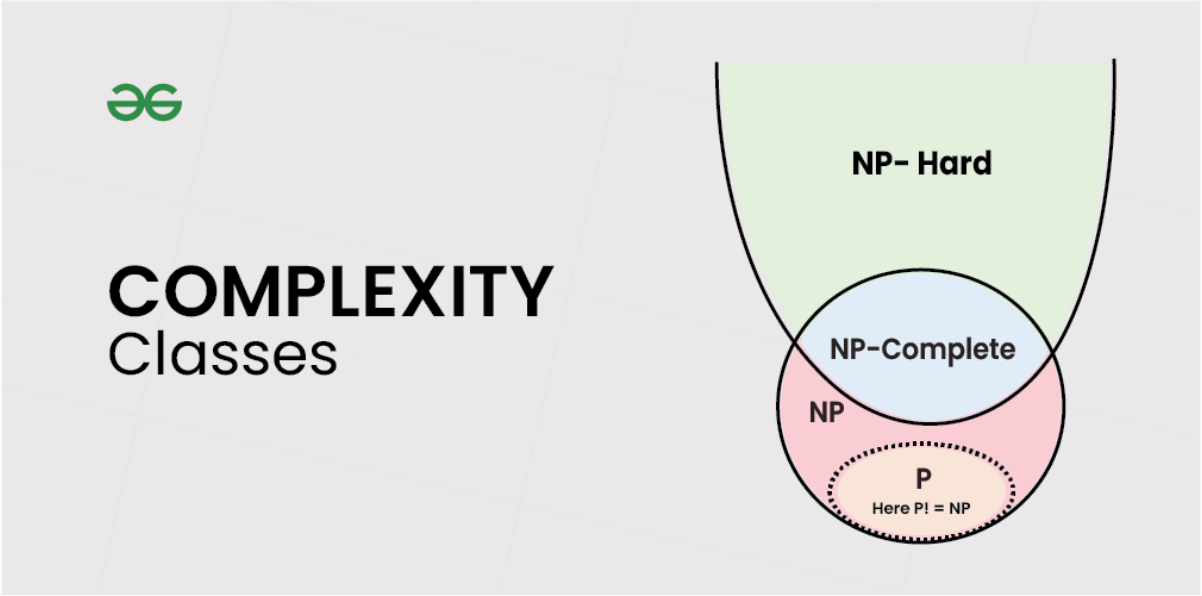
In computer science, problems are divided into classes known as **Complexity Classes**. In complexity theory, a Complexity Class is a set of problems with related complexity. With the help of complexity theory, we try to cover the following.

- Problems that cannot be solved by computers.
- Problems that can be efficiently solved (solved in Polynomial time) by computers.
- Problems for which no efficient solution (only exponential time algorithms) exist.

The common resources required by a solution are are time and space, meaning how much time the algorithm takes to solve a problem and the corresponding memory usage.

- The time complexity of an algorithm is used to describe the number of steps required to solve a problem, but it can also be used to describe how long it takes to verify the answer.
- The space complexity of an algorithm describes how much memory is required for the algorithm to operate.
- An algorithm having time complexity of the form $O(n^k)$ for input n and constant k is called polynomial time solution. These solutions scale well. On the other hand, time complexity of the form $O(k^n)$ is exponential time.

Complexity classes are useful in organizing similar types of problems.



Types of Complexity Classes

This article discusses the following complexity classes:

P Class

The P in the P class stands for **Polynomial Time.** It is the collection of decision problems(problems with a "yes" or "no" answer) that can be solved by a deterministic machine (our computers) in polynomial time.

Features:

- The solution to **P problem**s is easy to find.
- **P** is often a class of computational problems that are solvable and tractable. Tractable means that the problems can be solved in theory as well as in practice. But the problems that can be solved in theory but not in practice are known as intractable.

Most of the coding problems that we solve fall in this category like the below.

1. Calculating the greatest common divisor.
2. Finding a maximum matching.
3. Merge Sort

NP Class

The NP in NP class stands for **Non-deterministic Polynomial Time**. It is the collection of decision problems that can be solved by a non-deterministic machine (note that our computers are deterministic) in polynomial time.

Features:

- The solutions of the NP class might be hard to find since they are being solved by a non-deterministic machine but the solutions are easy to verify.
- Problems of NP can be verified by a deterministic machine in polynomial time.

Example:

Let us consider an example to better understand the **NP class**.
Suppose there is a company having a total of **1000** employees having unique employee **IDs**. Assume that there are **200** rooms available for them. A selection of **200** employees must be paired together, but the CEO of the company has the data of some employees who can't work in the same room due to personal reasons.
This is an example of an **NP** problem. Since it is easy to check if the given choice of **200** employees proposed by a coworker is satisfactory or not i.e. no pair taken from the coworker list appears on the list given by the CEO. But generating such a list from scratch seems to be so hard as to be completely impractical.

It indicates that if someone can provide us with the solution to the problem, we can find the correct and incorrect pair in polynomial time. Thus for the **NP** class problem, the answer is possible, which can be calculated in polynomial time.

This class contains many problems that one would like to be able to solve effectively:

1. Boolean Satisfiability Problem (SAT).
2. Hamiltonian Path Problem.
3. Graph coloring.

Co-NP Class

Co-NP stands for the complement of NP Class. It means if the answer to a problem in Co-NP is No, then there is proof that can be checked in polynomial time.

Features:

- If a problem X is in NP, then its complement X' is also in CoNP.
- For an NP and CoNP problem, there is no need to verify all the answers at once in polynomial time, there is a need to verify only one particular answer "yes" or "no" in polynomial time for a problem to be in NP or CoNP.

Some example problems for CoNP are:

1. To check prime number.
2. Integer Factorization.

NP-hard class

An NP-hard problem is at least as hard as the hardest problem in NP and it is a class of problems such that every problem in NP reduces to NP-hard.

Features:

- All NP-hard problems are not in NP.
- It takes a long time to check them. This means if a solution for an NP-hard problem is given then it takes a long time to check whether it is right or not.
- A problem A is in NP-hard if, for every problem L in NP, there exists a polynomial-time reduction from L to A.

Some of the examples of problems in Np-hard are:

1. **Halting problem.**
2. **Qualified Boolean formulas.**
3. **No Hamiltonian cycle.**

NP-complete class

A problem is NP-complete if it is both NP and NP-hard. NP-complete problems are the hard problems in NP.

Features:

- NP-complete problems are special as any problem in NP class can be transformed or reduced into NP-complete problems in polynomial time.
- If one could solve an NP-complete problem in polynomial time, then one could also solve any NP problem in polynomial time.

Some example problems include:

1. Hamiltonian Cycle.
2. Satisfiability.
3. Vertex cover.

Complexity Class	Characteristic feature
P	Easily solvable in polynomial time.
NP	Yes, answers can be checked in polynomial time.
Co-NP	No, answers can be checked in polynomial time.
NP-hard	All NP-hard problems are not in NP and it takes a long time to check them.

NP-complete	A problem that is NP and NP-hard is NP-complete.

Next Article

Can Run Time Complexity of a comparison-based sorting algorithm be less than N logN?

Similar Reads

Analysis of Algorithms

Analysis of Algorithms is a fundamental aspect of computer science that involves evaluating performance of algorithms and programs. Efficiency is measured in terms of time and space.Basic...

1 min read

Complete Guide On Complexity Analysis - Data Structure and Algorithms Tutorial

Complexity analysis is defined as a technique to characterise the time taken by an algorithm with respect to input size (independent from the machine, language and compiler). It is used for...

15+ min read

Why is Analysis of Algorithm important?

Why is Performance of Algorithms Important ? There are many important things that should be taken care of, like user-friendliness, modularity, security, maintainability, etc. Why worry about...

2 min read

Types of Asymptotic Notations in Complexity Analysis of Algorithms

We have discussed Asymptotic Analysis, and Worst, Average, and Best Cases of Algorithms. The main idea of asymptotic analysis is to have a measure of the efficiency of algorithms that don't...

8 min read

Worst, Average and Best Case Analysis of Algorithms

In the previous post, we discussed how Asymptotic analysis overcomes the problems of the naive way of analyzing algorithms. Now let us learn about What is Worst, Average, and Best cases of a...

10 min read

Asymptotic Analysis

Given two algorithms for a task, how do we find out which one is better? One naive way of doing this is - to implement both the algorithms and run the two programs on your computer for differe...

3 min read

How to Analyse Loops for Complexity Analysis of Algorithms

We have discussed Asymptotic Analysis, Worst, Average and Best Cases and Asymptotic Notations in previous posts. In this post, an analysis of iterative programs with simple examples i...

15+ min read

Sample Practice Problems on Complexity Analysis of Algorithms

Prerequisite: Asymptotic Analysis, Worst, Average and Best Cases, Asymptotic Notations, Analysis of loops.Problem 1: Find the complexity of the below recurrence: { 3T(n-1), if n>0,T(n) = { 1,...

14 min read

Basics on Analysis of Algorithms

Asymptotic Notations

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305